

DATABASE SCHEMA DOCUMENTATION

Compliance Management Platform

RAG-Optimised Schema Knowledge Base · Vector DB Ready

Version	v3.0 — POC / RAG-Optimised
Date	March 04, 2026
Classification	Internal / Confidential
Purpose	Vector DB Ingestion · RAG Query Generation · SQL Auto-Generation
Status	Proof of Concept (POC)
RAG Fix	v3 adds chunk-safe [TABLE][SECTION] metadata tags, full column text, clean ER map, NLP keyword badges per table

Every section of this document begins with a machine-readable [TABLE] [SECTION] tag so that any chunk retrieved from the vector DB is self-identifying. Column definitions are fully expanded with no truncation. The ER relationship map uses plain text for reliable ingestion.

[TABLE: DOCUMENT] [SECTION: table_of_contents] [DOC: compliance_platform_schema_v3]

Table of Contents	
1.	System Overview & Architecture
2.	Entity Relationship Summary
3.	Table: organizations
4.	Table: users
5.	Table: frameworks
6.	Table: controls
7.	Table: policies
8.	Table: audits
9.	Table: audit_findings
10.	Table: risks
11.	Table: tasks
12.	Table: evidence
13.	Foreign Key & Relationship Map
14.	Indexes, Constraints & Generated Columns
15.	Data Flow Through the System
16.	Example SQL Queries
17.	RAG Retrieval Hints, Query Rules & Glossary

[TABLE: DOCUMENT] [SECTION: system_overview_and_architecture] [DOC: compliance_platform_schema_v3]

1. System Overview & Architecture

This database underpins a **compliance management platform** that enables organizations to track regulatory frameworks (ISO 27001, SOC 2, HIPAA), manage full audit lifecycles, maintain security policies, monitor information security risks, assign remediation tasks, and store supporting evidence. The schema is a relational **PostgreSQL** design and acts as the single source of truth for all compliance-related activities across multiple client organizations.

1.1 Architectural Principles

MULTI-TENANT Every major entity table carries an **org_id** foreign key ensuring data isolation per organization. All queries MUST be scoped with WHERE org_id = ?.

Principle	Description
Multi-tenancy	org_id present in organizations, users, policies, audits, risks, tasks. Each org's data is fully isolated.
Framework-driven	Compliance structured as Frameworks → Controls → Policies/Audits — fully traceable hierarchy.
Audit-centric	Audits produce Findings; Findings trigger Tasks; Tasks are evidenced by uploaded Evidence documents.
Risk register	Risks are scored automatically (likelihood x impact, max 25) and linked to Controls and Owners.
RBAC	Every user has a role (admin, auditor, analyst, viewer) governing access permissions.
Immutable audit trail	created_at / uploaded_at timestamps use DEFAULT NOW() — automatic insert-time audit trail.

1.2 Database Technology Stack

- Engine: PostgreSQL
- Primary keys: SERIAL (auto-incrementing integer, never null)
- Enumerations: CHECK constraints on VARCHAR fields (no separate ENUM types used)
- Computed columns: GENERATED ALWAYS AS (expression) STORED — for risk_score
- Timestamps: DEFAULT NOW() on created_at and uploaded_at fields

1.3 Table Inventory

#	Table Name	Primary Business Purpose	Scale
1	organizations	Tenant registry — one row per client organization	Low (tens)
2	users	Platform users with org membership and RBAC role	Hundreds
3	frameworks	Reference list of compliance frameworks (ISO 27001, SOC 2...)	Low (<20)
4	controls	Individual control requirements within each framework	Hundreds
5	policies	Org-level policy documents mapped to controls	Hundreds
6	audits	Audit engagements: org + framework + auditor + dates	Hundreds
7	audit_findings	Deficiencies found during audits against specific controls	Thousands
8	risks	Risk register with auto-calculated risk_score	Thousands

#	Table Name	Primary Business Purpose	Scale
9	tasks	Remediation action items linked to findings or risks	Thousands
10	evidence	Metadata for files uploaded as compliance proof	Thousands

[TABLE: DOCUMENT] [SECTION: entity_relationship_summary] [DOC: compliance_platform_schema_v3]

2. Entity Relationship Summary

All foreign-key relationships between the 10 tables. Direction: child references parent.

```

TABLE RELATIONSHIPS (child --> parent)
users .org_id --> organizations.org_id
controls .framework_id --> frameworks.framework_id
policies .org_id --> organizations.org_id
policies .control_id --> controls.control_id
audits .org_id --> organizations.org_id
audits .framework_id --> frameworks.framework_id
audit_findings .audit_id --> audits.audit_id
audit_findings .control_id --> controls.control_id
risks .org_id --> organizations.org_id
risks .control_id --> controls.control_id
risks .owner_id --> users.user_id
tasks .org_id --> organizations.org_id
tasks .assigned_to --> users.user_id
tasks .finding_id --> audit_findings.finding_id
tasks .risk_id --> risks.risk_id
evidence .task_id --> tasks.task_id
evidence .audit_id --> audits.audit_id
evidence .uploaded_by --> users.user_id

```

2.1 Cardinality Summary

Relationship	Cardinality	Notes
organizations -> users	1 : N	One org has many users
organizations -> policies	1 : N	One org owns many policies
organizations -> audits	1 : N	One org undergoes many audits
organizations -> risks	1 : N	One org has many risks
organizations -> tasks	1 : N	One org has many tasks
frameworks -> controls	1 : N	One framework contains many controls
frameworks -> audits	1 : N	One framework is used in many audits
controls -> policies	1 : N	One control addressed by many policies
controls -> audit_findings	1 : N	One control can have many findings
controls -> risks	1 : N	One control linked to many risks
audits -> audit_findings	1 : N	One audit produces many findings
audits -> evidence	1 : N	One audit supported by many evidence files
audit_findings -> tasks	1 : N	One finding triggers many tasks
risks -> tasks	1 : N	One risk triggers many mitigation tasks
tasks -> evidence	1 : N	One task supported by many evidence files
users -> policies	1 : N	One user owns many policies (owner_id)
users -> audits	1 : N	One user conducts many audits (auditor_id)
users -> tasks	1 : N	One user assigned many tasks (assigned_to)
users -> evidence	1 : N	One user uploads many evidence files (uploaded_by)

2.2 Key Navigational Paths

COMPLIANCE PATH	frameworks --> controls --> policies --> audits --> audit_findings --> tasks --> evidence
RISK PATH	organizations --> risks (via control gap) --> tasks --> evidence
ACCOUNTABILITY PATH	users referenced as owner_id / auditor_id / assigned_to / uploaded_by across 5 tables

3. Table: organizations

[TABLE: organizations] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS organization, company, client, tenant, org_id, industry, sector, country

PURPOSE Top-level tenant table. Every row is one client organization. All other entities reference this via org_id to enforce multi-tenant data isolation.

Business Context

When a company onboards, a row is inserted here. The org_id is stamped onto every user, policy, audit, risk, and task for that company. SQL queries must almost always include WHERE org_id = ? to scope to a single tenant. The industry and country fields support benchmarking and jurisdiction-specific filtering.

[TABLE: organizations] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
org_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented tenant identifier. Root anchor for all tenant-scoped data.
name	VARCHAR(150)	NOT NULL	—	Full legal or trading name of the organization (e.g., 'Acme Corp').
industry	VARCHAR(100)	YES	—	Industry sector: Finance, Technology, Healthcare, etc. Used for cross-organization benchmarking.
country	VARCHAR(100)	YES	—	Country of incorporation. Used for jurisdiction-specific compliance rule filtering.
created_at	TIMESTAMP	NOT NULL	DEFAULT NOW()	Timestamp auto-set when the organization was registered. Immutable after creation.

[TABLE: organizations] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** org_id — SERIAL, auto-increment, never null
- name is NOT NULL — an organization cannot be created without a name.
- created_at defaults to current DB timestamp at insert — cannot be omitted.
- No UNIQUE constraint on name — org_id is the canonical unique tenant identifier.

[TABLE: organizations] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

org_id	name	industry	country	created_at
1	Acme Corp	Finance	USA	2025-01-10 09:00:00
2	TechNova Ltd	Technology	UK	2025-01-10 09:05:00
3	MedSafe Inc	Healthcare	Canada	2025-01-10 09:10:00

[TABLE: organizations] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

Retrieve all organizations in the Finance industry

```

SELECT org_id, name, country FROM organizations WHERE industry = 'Finance';

```

Resolve an org name to org_id before querying any child table

```

SELECT org_id FROM organizations WHERE name = 'MedSafe Inc';

```

Count organizations per country

```

SELECT country, COUNT(*) AS org_count FROM organizations GROUP BY country ORDER BY org_count DESC;

```

[TABLE: organizations] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'which company', 'client', 'organization name', 'tenant', 'org_id' — start here to resolve org names to IDs.
- Always resolve a human-readable org name to org_id via this table before querying any child table.
- industry and country columns support sector/jurisdiction filtering.

4. Table: users

[TABLE: users] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS user, person, employee, who, owner, auditor, analyst, admin, viewer, role, email, name, assigned to

PURPOSE Stores all platform users across all organizations. Each user belongs to exactly one org and has a role governing what they can do.

Business Context

Users appear throughout the schema as policy owners, audit conductors, risk owners, task assignees, and evidence uploaders. The role field drives RBAC: admin configures the platform, auditor conducts audits and raises findings, analyst manages risks and policies, viewer has read-only access. email is globally unique — the login identity.

[TABLE: users] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
user_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented user identifier.
org_id	INT	NOT NULL	FK --> organizations(org_id)	Organization this user belongs to. Enforces tenant data isolation.
full_name	VARCHAR(150)	NOT NULL	—	User's full display name (first + last). Used for human-readable attribution.
email	VARCHAR(150)	NOT NULL	UNIQUE	Login email. Globally unique across ALL tenants on the platform.
role	VARCHAR(50)	YES	CHECK IN ('admin','auditor','analyst','viewer')	Platform RBAC role. Governs what actions the user can perform.
created_at	TIMESTAMP	NOT NULL	DEFAULT NOW()	Account creation timestamp. Auto-set on insert.

[TABLE: users] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** user_id — SERIAL, auto-increment
- Foreign Key:** org_id --> organizations(org_id) — every user must belong to a registered organization
- email must be globally unique across all organizations (UNIQUE constraint).
- role is CHECK-constrained to: 'admin', 'auditor', 'analyst', 'viewer' only.
- full_name and email are mandatory (NOT NULL).

[TABLE: users] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

user_id	org_id	full_name	email	role
1	1	Alice Johnson	alice@acme.com	admin
2	1	Bob Smith	bob@acme.com	auditor

user_id	org_id	full_name	email	role
3	2	Carol Williams	carol@technova.com	analyst
4	3	David Lee	david@medsafe.com	admin
5	3	Jabir Khan	jabir@medsafe.com	auditor

[TABLE: users] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

Find all auditors in a specific organization

```
SELECT u.full_name, u.email FROM users u WHERE u.org_id = 3 AND u.role = 'auditor';
```

List users with their organization names

```
SELECT u.full_name, u.role, o.name AS organization FROM users u JOIN organizations o ON u.org_id = o.org_id ORDER BY o.name, u.role;
```

Find all tasks assigned to a named user

```
SELECT t.title, t.due_date, t.status FROM tasks t JOIN users u ON t.assigned_to = u.user_id WHERE u.full_name = 'Jabir Khan';
```

[TABLE: users] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: person's name, 'who owns', 'assigned to', 'uploaded by', 'audited by' — always join through this table.
- Resolve user names to user_id before querying: policies.owner_id, audits.auditor_id, risks.owner_id, tasks.assigned_to, evidence.uploaded_by.
- Role-based queries: 'who are the admins at MedSafe?' --> filter users WHERE org_id = ? AND role = 'admin'.

5. Table: frameworks

[TABLE: frameworks] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS framework, standard, ISO 27001, SOC 2, HIPAA, regulation, certification, compliance norm

PURPOSE Reference table of compliance frameworks (ISO 27001, SOC 2, HIPAA). Shared across all tenants. Root of the compliance hierarchy.

Business Context

Frameworks are the top of the hierarchy: Framework --> Controls --> Policies/Audits. When an organization pursues a certification, they link their audits and controls to a framework_id. The version field differentiates editions (e.g., ISO 27001:2022 vs 2013).

[TABLE: frameworks] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
framework_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented framework identifier.
name	VARCHAR(100)	NOT NULL	—	Short official framework name: 'ISO 27001', 'SOC 2', 'HIPAA'.
version	VARCHAR(20)	YES	—	Edition string: '2022' for ISO 27001, 'Type II' for SOC 2, '2013' for HIPAA.
description	TEXT	YES	—	Human-readable scope summary. Ingested by RAG to understand framework details.

[TABLE: frameworks] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** framework_id — SERIAL, auto-increment
- name is NOT NULL — mandatory.
- No UNIQUE on name — version field differentiates multiple editions of the same standard.

[TABLE: frameworks] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

framework_id	name	version	description
1	ISO 27001	2022	Information security management standard
2	SOC 2	Type II	Service organization control for security and availability
3	HIPAA	2013	Health data privacy and security regulations

[TABLE: frameworks] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

List all available compliance frameworks

```

SELECT framework_id, name, version, description FROM frameworks ORDER BY name;

```

Find orgs being audited against ISO 27001

```

SELECT DISTINCT o.name AS organization FROM audits a JOIN organizations o ON a.org_id = o.org_id JOIN
frameworks f ON a.framework_id = f.framework_id WHERE f.name = 'ISO 27001';

```

[TABLE: frameworks] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'ISO 27001', 'SOC 2', 'HIPAA', 'compliance standard', 'regulation', 'framework' — query this table first.
- framework_id is referenced by both controls and audits — join here to resolve framework names to IDs.

6. Table: controls

[TABLE: controls] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS control, requirement, clause, domain, safeguard, control_code, category, governance, access control, data protection

PURPOSE Individual control requirements from each framework. The atomic unit of compliance — policies are written to meet controls, audits test them, risks arise from gaps in them.

Business Context

Each control belongs to one framework and has a standardized code. Policies address controls; audits produce findings against controls; risks are linked to control gaps. The category field groups controls by domain (Governance, Access Control, etc.).

[TABLE: controls] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
control_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented control identifier.
framework_id	INT	NOT NULL	FK --> frameworks	Parent framework. Every control must belong to a valid framework.
control_code	VARCHAR(30)	NOT NULL	—	Standardized code from the framework spec: 'ISO-A.5.1', 'SOC-CC6.1', 'H...
title	VARCHAR(200)	NOT NULL	—	Short title of the control requirement.
description	TEXT	YES	—	Full text of what the control requires organizations to implement.
category	VARCHAR(100)	YES	—	Domain grouping: Governance, Asset Management, Access Control, Mor...

[TABLE: controls] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** control_id — SERIAL, auto-increment
- Foreign Key:** framework_id --> frameworks(framework_id) — control must belong to a valid framework
- framework_id and control_code are both NOT NULL — mandatory.
- control_code is NOT globally unique — the same code can exist under different frameworks.

[TABLE: controls] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

control_id	framework	control_code	title	category
1	ISO 27001	ISO-A.5.1	Information Security Policies	Governance
2	ISO 27001	ISO-A.8.1	Asset Management	Asset Management
3	SOC 2	SOC-CC6.1	Logical Access Controls	Access Control

control_id	framework	control_code	title	category
4	SOC 2	SOC-CC7.2	Incident Monitoring	Monitoring
5	HIPAA	HIPAA-164.312	Technical Safeguards	Data Protection

[TABLE: controls] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

All controls for a specific framework

```
SELECT c.control_code, c.title, c.category FROM controls c JOIN frameworks f ON c.framework_id = f.framework_id WHERE f.name = 'HIPAA';
```

Controls that have open findings

```
SELECT DISTINCT c.control_code, c.title, f.name AS framework FROM controls c JOIN frameworks f ON c.framework_id = f.framework_id JOIN audit_findings af ON c.control_id = af.control_id WHERE af.status != 'closed';
```

[TABLE: controls] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'control requirement', 'clause', 'ISO-A.5.1', 'data protection controls', 'access control clause' — query here.
- Use control_code to cross-reference external framework documentation.
- category supports domain filtering: 'show all monitoring controls' --> WHERE category = 'Monitoring'.

7. Table: policies

[TABLE: policies] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS policy, procedure, document, written policy, review date, policy owner, draft, active, retired, overdue

PURPOSE Organization-level policy documents created to formally satisfy specific controls. Lifecycle: draft -> active -> retired.

Business Context

Each policy is owned by a user and linked to both a control and an organization. review_date tracks when it must be re-assessed. Audit findings can flag a policy as outdated or missing. Overdue policy reviews are a frequent compliance risk trigger.

[TABLE: policies] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
policy_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented policy identifier.
org_id	INT	NOT NULL	FK --> organizations(org_id)	Organization that owns this policy document.
control_id	INT	NOT NULL	FK --> controls(control_id)	The compliance control this policy is written to address and satisfy.
title	VARCHAR(200)	NOT NULL	—	Descriptive title of the policy document.
status	VARCHAR(30)	YES	CHECK IN ('draft','active','retired')	Lifecycle stage. Active = approved and in force. Draft = pending approval.
owner_id	INT	YES	FK --> users(user_id)	User accountable for maintaining and reviewing this policy. May be NULL.
review_date	DATE	YES	—	Next scheduled review/approval date. Compare with CURRENT_DATE for overdue.

[TABLE: policies] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** policy_id — SERIAL, auto-increment
- Foreign Key:** org_id --> organizations(org_id)
- Foreign Key:** control_id --> controls(control_id)
- Foreign Key:** owner_id --> users(user_id) [NULLABLE]
- status must be one of: 'draft', 'active', 'retired' — CHECK constraint enforced at DB level.
- review_date is not DB-enforced for overdue logic — application monitors it.

[TABLE: policies] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

policy_id	org	control	title	status	owner	review_date
1	Acme	ISO-A.5.1	Acme Information Security Policy	active	Alice Johnson	2025-06-01

policy_id	org	control	title	status	owner	review_date
2	Acme	SOC-CC6.1	Acme Access Control Policy	active	Bob Smith	2025-09-01
3	MedSafe	HIPAA-164.312	MedSafe ePHI Technical Safeguards	draft	David Lee	2025-12-01
4	TechNova	ISO-A.8.1	TechNova Asset Inventory Policy	active	Carol Williams	2026-01-15

[TABLE: policies] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

Active policies for an org with owner and control details

```
SELECT p.title, c.control_code, u.full_name AS owner, p.review_date FROM policies p JOIN controls c ON p.control_id = c.control_id JOIN users u ON p.owner_id = u.user_id WHERE p.org_id = 1 AND p.status = 'active';
```

Active policies overdue for review

```
SELECT p.title, p.review_date, o.name AS org, u.full_name AS owner FROM policies p JOIN organizations o ON p.org_id = o.org_id JOIN users u ON p.owner_id = u.user_id WHERE p.review_date < CURRENT_DATE AND p.status = 'active' ORDER BY p.review_date;
```

Draft policies pending approval

```
SELECT p.title, u.full_name AS owner, o.name AS organization FROM policies p JOIN users u ON p.owner_id = u.user_id JOIN organizations o ON p.org_id = o.org_id WHERE p.status = 'draft';
```

[TABLE: policies] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'policy status', 'who owns a policy', 'overdue review', 'draft policy', 'policy document' — query here.
- JOIN controls to answer: 'which policies cover HIPAA data protection requirements'.
- review_date < CURRENT_DATE AND status = 'active' --> overdue policy review detection.

8. Table: audits

[TABLE: audits] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS audit, engagement, assessment, review, certification, auditor, audit status, planned, in progress, completed

PURPOSE Formal audit engagements conducted by an auditor against an organization's compliance with a specific framework. Lifecycle: planned -> in_progress -> completed.

Business Context

All findings, tasks, and evidence trace back to an audit. auditor_id identifies the platform user conducting the work. Audit status drives the workflow: planned --> in_progress --> completed (or cancelled if abandoned).

[TABLE: audits] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
audit_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented audit identifier.
org_id	INT	NOT NULL	FK --> organizations(org_id)	Organization being audited.
framework_id	INT	NOT NULL	FK --> frameworks(framework_id)	Framework against which the audit is being conducted.
auditor_id	INT	YES	FK --> users(user_id)	User (auditor role) responsible for conducting the audit. May be NULL.
audit_name	VARCHAR(200)	YES	—	Human-readable title of the audit engagement.
start_date	DATE	YES	—	Official start date of the audit.
end_date	DATE	YES	—	Planned or actual end date. Should be >= start_date (application-enforced).
status	VARCHAR(30)	YES	CHECK IN (planned in_progress completed cancelled)	Current lifecycle stage of the audit.

[TABLE: audits] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** audit_id — SERIAL, auto-increment
- Foreign Key:** org_id --> organizations(org_id)
- Foreign Key:** framework_id --> frameworks(framework_id)
- Foreign Key:** auditor_id --> users(user_id) [NULLABLE]
- status must be: 'planned', 'in_progress', 'completed', or 'cancelled'.
- end_date >= start_date enforced at application layer only.

[TABLE: audits] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

audit_id	org	framework	auditor	audit_name	start_date	end_date	status
1	Acme	ISO 27001	Bob Smith	Acme ISO 27001 2024-10-01 Audit	2024-10-01	2024-11-15	completed
2	MedSafe	HIPAA	Jabir Khan	MedSafe HIPAA 2025-01-10 Audit	2025-01-10	2025-02-28	in_progress
3	TechNova	SOC 2	Carol Williams	TechNova SOC 2 2025-03-01 Review	2025-03-01	2025-04-30	planned

[TABLE: audits] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

All in-progress audits with auditor details

```

SELECT a.audit_name, o.name AS org, f.name AS framework, u.full_name AS auditor FROM audits a JOIN
organizations o ON a.org_id = o.org_id JOIN frameworks f ON a.framework_id = f.framework_id JOIN users u
ON a.auditor_id = u.user_id WHERE a.status = 'in_progress';
    
```

Completed audits with total finding counts

```

SELECT a.audit_name, COUNT(af.finding_id) AS total_findings FROM audits a LEFT JOIN audit_findings af ON
a.audit_id = af.audit_id WHERE a.status = 'completed' GROUP BY a.audit_id, a.audit_name;
    
```

[TABLE: audits] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'audit status', 'who is auditing', 'ongoing audits', 'planned audit', 'completed audit' — query here.
- JOIN audit_findings to count findings per audit.
- auditor_id --> users: always join to get the auditor's name.

9. Table: audit_findings

[TABLE: audit_findings] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS finding, gap, deficiency, non-conformity, issue, breach, severity, critical, open, in remediation, closed

PURPOSE Deficiencies, gaps, or non-conformities found during an audit against a specific control. Drives the remediation workflow by spawning tasks.

Business Context

Findings are the primary output of audits. When a control is not properly implemented, the auditor raises a finding here. Findings spawn Tasks for remediation. A finding is only 'closed' when sufficient evidence is provided. Critical/high severity findings require immediate action.

[TABLE: audit_findings] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
finding_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented finding identifier.
audit_id	INT	NOT NULL	FK --> audits(audit_id)	Audit during which this finding was raised.
control_id	INT	NOT NULL	FK --> controls(control_id)	The specific control found to be deficient or not properly implemented.
severity	VARCHAR(20)	YES	CHECK IN ('low','medium','high','critical')	Criticality: low / medium / high / critical.
title	VARCHAR(200)	YES	—	Short title describing the deficiency.
description	TEXT	YES	—	Full description of what was found and why it is a deficiency.
status	VARCHAR(30)	YES	CHECK IN ('open','in_remediation','closed')	Remediation status: open = unaddressed, in_remediation = being fixed, closed = resolved.

[TABLE: audit_findings] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** finding_id — SERIAL, auto-increment
- Foreign Key:** audit_id --> audits(audit_id)
- Foreign Key:** control_id --> controls(control_id)
- severity: 'low', 'medium', 'high', 'critical' — CHECK constraint.
- status: 'open', 'in_remediation', 'closed' — CHECK constraint.
- Open items: status IN ('open', 'in_remediation').

[TABLE: audit_findings] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

finding_id	audit	control	severity	title	status
1	Acme ISO 27001	ISO-A.5.1	medium	Policy not reviewed annually	open

finding_id	audit	control	severity	title	status
2	Acme ISO 27001	ISO-A.8.1	high	Incomplete asset inventory	in_remediation
3	MedSafe HIPAA	HIPAA-164.312	critical	ePHI encryption gap	open

[TABLE: audit_findings] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

All open/critical findings across the platform

```

SELECT af.title, af.severity, a.audit_name, o.name AS org FROM audit_findings af JOIN audits a ON
af.audit_id = a.audit_id JOIN organizations o ON a.org_id = o.org_id WHERE af.status != 'closed' AND
af.severity IN ('high', 'critical') ORDER BY af.severity DESC;

```

Finding count by severity for a specific org

```

SELECT af.severity, COUNT(*) AS count FROM audit_findings af JOIN audits a ON af.audit_id = a.audit_id
WHERE a.org_id = 3 GROUP BY af.severity ORDER BY COUNT(*) DESC;

```

[TABLE: audit_findings] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'open findings', 'critical issues', 'audit gaps', 'non-conformities', 'deficiencies' — query here.
- JOIN audits + organizations to scope findings to a specific org.
- status IN ('open', 'in_remediation') = items still requiring action.

10. Table: risks

[TABLE: risks] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS risk, threat, exposure, likelihood, impact, risk score, risk register, open risk, mitigated, accepted

PURPOSE Risk register. Each row is an identified risk for an organization, optionally linked to a control gap. risk_score is AUTO-COMPUTED as likelihood x impact (max 25). CANNOT be set manually.

Business Context

Risks are raised when a control gap or threat is recognized. risk_score (max 25) determines remediation priority. Risk owners are accountable for mitigation. Lifecycle: open --> mitigated / accepted --> closed. Tasks are created to address open risks.

[TABLE: risks] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
risk_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented risk identifier.
org_id	INT	NOT NULL	FK --> organizations(org_id)	Organization carrying this risk.
control_id	INT	YES	FK --> controls(control_id) [NULLABLE]	Control gap gives rise to this risk. Optional — a risk may be free-standing.
title	VARCHAR(200)	YES	—	Short title of the risk scenario.
likelihood	INT	YES	CHECK BETWEEN 1 AND 5	Probability of occurrence. Scale: 1=Very Low, 2=Low, 3=Medium, 4=High, 5=Critical
impact	INT	YES	CHECK BETWEEN 1 AND 5	Business impact if realized. Scale: 1=Negligible, 2=Minor, 3=Moderate, 4=Major, 5=Critical
risk_score	INT	NOT NULL	GENERATED ALWAYS AS (likelihood * impact) STORED	AUTO-COMPUTED RISK SCORE (0-25). Physically stored. CANNOT be set via UI.
owner_id	INT	YES	FK --> users(user_id) [NULLABLE]	User accountable for monitoring and mitigating this risk.
status	VARCHAR(30)	YES	CHECK IN ('open','mitigated','accepted','closed')	Risk disposition: open / mitigated / accepted / closed.

[TABLE: risks] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- **Primary Key:** risk_id — SERIAL, auto-increment
- **Foreign Key:** org_id --> organizations(org_id)
- **Foreign Key:** control_id --> controls(control_id) [NULLABLE]
- **Foreign Key:** owner_id --> users(user_id) [NULLABLE]
- likelihood and impact must each be INTEGER between 1 and 5 — CHECK constraint.
- risk_score is GENERATED ALWAYS AS (likelihood * impact) STORED — physically persisted, auto-updated, NEVER manually insertable or updatable.
- status: 'open', 'mitigated', 'accepted', 'closed' — CHECK constraint.
- Score bands: Critical >= 20 | High 15-19 | Medium 10-14 | Low 5-9 | Very Low < 5.

[TABLE: risks] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

risk_id	org	control	title	L	I	Score	status
1	Acme	SOC-CC6.1	Unauthorized access to finance system	5	5	20	open
2	MedSafe	HIPAA-164.312	PHI data breach via unencrypted backup	4	4	15	open
3	TechNova	SOC-CC7.2	Delayed incident response	2	3	6	mitigated

[TABLE: risks] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

High-priority open risks (score >= 10)

```

SELECT r.title, r.risk_score, r.likelihood, r.impact, u.full_name AS owner, o.name AS org FROM risks r
JOIN organizations o ON r.org_id = o.org_id JOIN users u ON r.owner_id = u.user_id WHERE r.status =
'open' AND r.risk_score >= 10 ORDER BY r.risk_score DESC;
    
```

Risk dashboard by organization

```

SELECT o.name, COUNT(*) AS total_risks, SUM(CASE WHEN r.status='open' THEN 1 ELSE 0 END) AS open_risks,
MAX(r.risk_score) AS highest_score FROM risks r JOIN organizations o ON r.org_id = o.org_id GROUP BY
o.name ORDER BY highest_score DESC NULLS LAST;
    
```

[TABLE: risks] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'risk score', 'highest risk', 'open risks', 'likelihood', 'impact', 'risk register' — query here.
- CRITICAL: risk_score is GENERATED — NEVER use INSERT/UPDATE on it. Use WHERE risk_score >= X for reads.
- Score bands for query generation: >= 20 critical | 15-19 high | 10-14 medium | 5-9 low | < 5 very low.

11. Table: **tasks**

[TABLE: tasks] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS task, action item, remediation, ticket, overdue, SLA, due date, priority, assigned to, in progress, done

PURPOSE Remediation action items assigned to users, triggered by audit findings or risks. Tracked with due dates and priority for SLA management.

Business Context

Tasks are the operational backbone of remediation. finding_id and risk_id are both nullable — a task may relate to a finding, a risk, both, or neither. Completion (status='done') is typically accompanied by uploading evidence.

[TABLE: tasks] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
task_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented task identifier.
org_id	INT	NOT NULL	FK --> organizations(org_id)	Organization this task belongs to.
assigned_to	INT	YES	FK --> users(user_id) [NULLABLE]	User responsible for completing this task. May be unassigned (NULL).
finding_id	INT	YES	FK --> audit_findings(finding_id) [NULLABLE]	Finding that triggered this task. NULL if task is risk-only.
risk_id	INT	YES	FK --> risks(risk_id) [NULLABLE]	Risk that triggered this task. NULL if task is finding-only.
title	VARCHAR(200)	YES	—	Short description of the action required.
due_date	DATE	YES	—	Target completion date for SLA tracking.
priority	VARCHAR(20)	YES	CHECK IN (low medium high)	Urgency level: low / medium / high.
status	VARCHAR(30)	YES	CHECK IN (todo in_progress done)	Completion status: todo / in_progress / done.

[TABLE: tasks] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- Primary Key:** task_id — SERIAL, auto-increment
- Foreign Key:** org_id --> organizations(org_id)
- Foreign Key:** assigned_to --> users(user_id) [NULLABLE]
- Foreign Key:** finding_id --> audit_findings(finding_id) [NULLABLE]
- Foreign Key:** risk_id --> risks(risk_id) [NULLABLE]
- priority: 'low', 'medium', 'high' — CHECK constraint.
- status: 'todo', 'in_progress', 'done' — CHECK constraint.
- finding_id and risk_id are both nullable — a task may relate to one, both, or neither.
- Overdue: WHERE due_date < CURRENT_DATE AND status != 'done'.

[TABLE: tasks] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

task_id	org	assigned_to	finding	risk	title	due_date	priority	status
1	Acme	Bob Smith	Incomplete asset	Access risk	Complete cloud asset inventory	2025-10-01	high	in_progress
2	MedSafe	Jabir Khan	ePHI encryption gap	Privacy risk	Encrypt all ePHI data files	2025-08-15	high	todo
3	Acme	Alice Johnson	Policy not reviewed	NULL	Update information security policy	2025-05-01	medium	done

[TABLE: tasks] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

Overdue open tasks across the platform

```
SELECT t.title, t.due_date, u.full_name AS assignee, o.name AS org FROM tasks t JOIN users u ON
t.assigned_to = u.user_id JOIN organizations o ON t.org_id = o.org_id WHERE t.status != 'done' AND
t.due_date < CURRENT_DATE ORDER BY t.due_date;
```

All tasks assigned to a specific user

```
SELECT t.title, t.priority, t.due_date, t.status FROM tasks t JOIN users u ON t.assigned_to = u.user_id
WHERE u.full_name = 'Jabir Khan' ORDER BY t.due_date;
```

Task completion rate per organization

```
SELECT o.name, COUNT(*) AS total_tasks, SUM(CASE WHEN t.status = 'done' THEN 1 ELSE 0 END) AS completed
FROM tasks t JOIN organizations o ON t.org_id = o.org_id GROUP BY o.name;
```

[TABLE: tasks] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'pending tasks', 'overdue items', 'who is responsible', 'remediation progress', 'action items' — query here.
- JOIN finding_id --> audit_findings to trace why a task was created.
- JOIN risk_id --> risks to understand the underlying risk being mitigated.

12. Table: **evidence**

[TABLE: evidence] [SECTION: overview_and_purpose] [DOC: compliance_platform_schema_v3]

NLP KEYWORDS evidence, proof, file, document, upload, attachment, supporting material, file name, file type

PURPOSE Metadata catalog for files uploaded as compliance proof. Actual files stored in external object storage (e.g., S3). This table holds only metadata and traceability links.

Business Context

Evidence is the documentary proof of compliance. When a task is done or an audit needs supporting documentation, users upload files and record metadata here. Both task_id and audit_id are nullable — evidence can relate to one, both, or neither. uploaded_by creates an accountability trail.

[TABLE: evidence] [SECTION: column_definitions] [DOC: compliance_platform_schema_v3]

Column Definitions

Column Name	Data Type	Nullable	Constraint / Default	Business Meaning
evidence_id	SERIAL	NOT NULL	PRIMARY KEY	Unique auto-incremented evidence record identifier.
task_id	INT	YES	FK --> tasks(task_id) [NULLABLE]	Task this evidence supports. NULL if uploaded at audit level only.
audit_id	INT	YES	FK --> audits(audit_id) [NULLABLE]	Audit this evidence is filed against. NULL if task-level only.
uploaded_by	INT	YES	FK --> users(user_id) [NULLABLE]	User who uploaded the file — accountability trail.
file_name	VARCHAR(255)	YES	—	Original file name including extension (e.g., 'policy_v2.pdf'). Not unique —
file_type	VARCHAR(50)	YES	—	File format abbreviation: PDF, XLSX, DOCX, PNG, etc.
description	TEXT	YES	—	Human-readable summary of what the file contains and its compliance re
uploaded_at	TIMESTAMP	NOT NULL	DEFAULT NOW()	Upload timestamp. Auto-set on insert. Immutable audit trail.

[TABLE: evidence] [SECTION: keys_and_constraints] [DOC: compliance_platform_schema_v3]

Keys, Relationships & Constraints

- **Primary Key:** evidence_id — SERIAL, auto-increment
- **Foreign Key:** task_id --> tasks(task_id) [NULLABLE]
- **Foreign Key:** audit_id --> audits(audit_id) [NULLABLE]
- **Foreign Key:** uploaded_by --> users(user_id) [NULLABLE]
- task_id and audit_id are both nullable — evidence can relate to one, both, or neither.
- file_name is NOT UNIQUE — multiple versions of the same document can be uploaded.
- uploaded_at auto-defaults to current timestamp — cannot be omitted.
- file content is NOT stored here — only metadata. Do NOT query this table for file contents.

[TABLE: evidence] [SECTION: sample_records] [DOC: compliance_platform_schema_v3]

Sample Records

evidence_id	task	audit	uploaded_by	file_name	type	description
1	Update IS policy	Acme ISO 27001	Alice Johnson	info_sec_policy_v2.pdf	PDF	Updated information security policy
2	Complete asset inv.	Acme ISO 27001	Bob Smith	cloud_asset_inventory.xlsx	XLSX	Partial cloud asset inventory spreadsheet
3	Encrypt ePHI	MedSafe HIPAA	Jabir Khan	encryption_config_report.pdf	PDF	Encryption gap analysis report

[TABLE: evidence] [SECTION: sql_query_examples] [DOC: compliance_platform_schema_v3]

SQL Query Examples

All evidence files for a specific audit

```
SELECT e.file_name, e.file_type, e.description, u.full_name AS uploaded_by, e.uploaded_at FROM evidence
e JOIN users u ON e.uploaded_by = u.user_id WHERE e.audit_id = 1 ORDER BY e.uploaded_at;
```

Evidence linked to completed tasks

```
SELECT t.title AS task, e.file_name, e.description, e.uploaded_at FROM evidence e JOIN tasks t ON
e.task_id = t.task_id WHERE t.status = 'done' ORDER BY e.uploaded_at DESC;
```

[TABLE: evidence] [SECTION: rag_retrieval_hints] [DOC: compliance_platform_schema_v3]

RAG Retrieval Hints

RETRIEVAL NOTE When these keywords or concepts appear in a user query, this table section should be retrieved.

- Keywords: 'uploaded documents', 'proof of compliance', 'supporting files', 'evidence', 'attachments' — query here.
- file_name and description are the primary searchable fields.
- This table stores METADATA only — no actual file content is stored in the database.

[TABLE: DOCUMENT]

[SECTION: foreign_key_and_relationship_map]

[DOC: compliance_platform_schema_v3]

13. Foreign Key & Relationship Map

Complete reference of all foreign key constraints. Use this to construct correct JOIN clauses.

Child Table	FK Column	Parent Table	PK Column	Nullable	Relationship Meaning
users	org_id	organizations	org_id	No	Every user belongs to exactly one organization
controls	framework_id	frameworks	framework_id	No	Every control belongs to exactly one framework
policies	org_id	organizations	org_id	No	Policy is owned by an organization
policies	control_id	controls	control_id	No	Policy addresses a specific control
policies	owner_id	users	user_id	Yes	Policy has an optional responsible owner (user)
audits	org_id	organizations	org_id	No	Audit is conducted for an organization
audits	framework_id	frameworks	framework_id	No	Audit is against a specific framework
audits	auditor_id	users	user_id	Yes	Audit is optionally assigned to an auditor user
audit_findings	audit_id	audits	audit_id	No	Finding raised during a specific audit
audit_findings	control_id	controls	control_id	No	Finding is against a specific control
risks	org_id	organizations	org_id	No	Risk belongs to an organization
risks	control_id	controls	control_id	Yes	Risk optionally linked to a control gap
risks	owner_id	users	user_id	Yes	Risk has an optional accountable owner
tasks	org_id	organizations	org_id	No	Task belongs to an organization
tasks	assigned_to	users	user_id	Yes	Task optionally assigned to a user
tasks	finding_id	audit_findings	finding_id	Yes	Task optionally triggered by an audit finding
tasks	risk_id	risks	risk_id	Yes	Task optionally triggered by a risk
evidence	task_id	tasks	task_id	Yes	Evidence optionally attached to a task
evidence	audit_id	audits	audit_id	Yes	Evidence optionally filed under an audit
evidence	uploaded_by	users	user_id	Yes	Evidence uploaded by a specific user

[TABLE: DOCUMENT] [SECTION: indexes_constraints_generated_columns] [DOC: compliance_platform_schema_v3]

14. Indexes, Constraints & Generated Columns

14.1 CHECK Constraints — Enumerated Values

Table	Column	Allowed Values	Business Meaning
users	role	admin auditor analyst viewer	Platform RBAC roles — 4 values only
policies	status	draft active retired	Policy lifecycle stages
audits	status	planned in_progress completed cancelled	Audit lifecycle — 4 stages
audit_findings	severity	low medium high critical	Finding risk severity levels
audit_findings	status	open in_remediation closed	Finding remediation status
risks	likelihood	INTEGER 1 to 5	Probability scale (1=Very Low, 5=Almost Certain)
risks	impact	INTEGER 1 to 5	Impact scale (1=Negligible, 5=Catastrophic)
risks	status	open mitigated accepted closed	Risk disposition lifecycle
tasks	priority	low medium high	Task urgency levels
tasks	status	todo in_progress done	Task completion status

14.2 UNIQUE Constraints

- users.email — Globally unique across ALL organizations. Serves as the login identity.

14.3 GENERATED Column

CRITICAL — **risks.risk_score** GENERATED ALWAYS AS (likelihood * impact) STORED. Physically stored on disk. Auto-updated when likelihood or impact change. Attempting to INSERT or UPDATE this column directly will raise a PostgreSQL error. Only use it in SELECT / WHERE / ORDER BY clauses.

14.4 DEFAULT Values

- organizations.created_at — DEFAULT NOW() (set at org insert time)
- users.created_at — DEFAULT NOW() (set at user account creation time)
- evidence.uploaded_at — DEFAULT NOW() (set when evidence record is inserted)

[TABLE: DOCUMENT] [SECTION: data_flow_through_the_system] [DOC: compliance_platform_schema_v3]

15. Data Flow Through the System

Understanding the two primary lifecycle flows is essential for generating correct multi-table SQL.

15.1 Compliance Audit Lifecycle

Step	Table(s)	Event / Action	Key Columns
1	organizations	Organization onboards to the platform	org_id created
2	frameworks	Admin selects target compliance framework	framework_id assigned
3	controls	System loads all controls for the framework	control_id per framework_id
4	policies	Org drafts and activates policies per control	policy: org_id + control_id + owner_id
5	audits	Auditor initiates formal audit engagement	audit: org_id + framework_id + auditor_id
6	audit_findings	Auditor raises findings against controls	finding: audit_id + control_id + severity
7	tasks	Remediation tasks created from findings	task: finding_id + assigned_to + due_date
8	evidence	User uploads proof of remediation	evidence: task_id + audit_id + uploaded_by
9	audit_findings	Finding status updated to closed	status = 'closed'
10	audits	Audit marked as completed	status = 'completed'

15.2 Risk Management Lifecycle

Step	Table(s)	Event / Action	Key Columns
1	risks	Risk identified, likelihood and impact set	risk_score auto-computed from likelihood * impact
2	risks	Risk owner assigned	owner_id --> users.user_id
3	tasks	Mitigation task created from risk	task: risk_id + assigned_to + due_date
4	evidence	Evidence uploaded after mitigation	evidence: task_id + uploaded_by
5	risks	Risk status updated to mitigated/accepted	status = 'mitigated' or 'accepted'

[TABLE: DOCUMENT] [SECTION: example_sql_queries] [DOC: compliance_platform_schema_v3]

16. Example SQL Queries

Multi-table query patterns for the RAG query generation component.

Q1: All open risks with score >= 10 for a specific organization

```
SELECT r.title, r.risk_score, r.likelihood, r.impact, c.control_code, c.title AS control_title,
u.full_name AS risk_owner FROM risks r JOIN organizations o ON r.org_id = o.org_id JOIN controls c ON
r.control_id = c.control_id JOIN users u ON r.owner_id = u.user_id WHERE o.name = 'MedSafe Inc' AND
r.status = 'open' AND r.risk_score >= 10 ORDER BY r.risk_score DESC;
```

Q2: All open findings with remediation tasks and assignees

```
SELECT af.title AS finding, af.severity, af.status AS finding_status, t.title AS task, t.due_date,
t.priority, u.full_name AS assignee FROM audit_findings af LEFT JOIN tasks t ON t.finding_id =
af.finding_id LEFT JOIN users u ON t.assigned_to = u.user_id WHERE af.status != 'closed' ORDER BY
af.severity DESC, t.due_date;
```

Q3: Active policies overdue for review with framework context

```
SELECT p.title AS policy, p.review_date, c.control_code, f.name AS framework, u.full_name AS owner,
o.name AS organization FROM policies p JOIN controls c ON p.control_id = c.control_id JOIN frameworks f
ON c.framework_id = f.framework_id JOIN users u ON p.owner_id = u.user_id JOIN organizations o ON
p.org_id = o.org_id WHERE p.review_date < CURRENT_DATE AND p.status = 'active' ORDER BY p.review_date;
```

Q4: All evidence files for a specific audit with related task context

```
SELECT e.file_name, e.file_type, e.description, u.full_name AS uploaded_by, e.uploaded_at, t.title AS
related_task FROM evidence e JOIN audits a ON e.audit_id = a.audit_id JOIN users u ON e.uploaded_by =
u.user_id LEFT JOIN tasks t ON e.task_id = t.task_id WHERE a.audit_name = 'MedSafe HIPAA Compliance
Audit' ORDER BY e.uploaded_at;
```

Q5: Full compliance dashboard — one row per organization

```
SELECT o.name AS organization, COUNT(DISTINCT a.audit_id) AS total_audits, COUNT(DISTINCT af.finding_id)
FILTER (WHERE af.status != 'closed') AS open_findings, COUNT(DISTINCT r.risk_id) FILTER (WHERE r.status
= 'open') AS open_risks, MAX(r.risk_score) AS highest_risk_score, COUNT(DISTINCT t.task_id) FILTER
(WHERE t.status != 'done') AS pending_tasks FROM organizations o LEFT JOIN audits a ON o.org_id =
a.org_id LEFT JOIN audit_findings af ON a.audit_id = af.audit_id LEFT JOIN risks r ON o.org_id =
r.org_id LEFT JOIN tasks t ON o.org_id = t.org_id GROUP BY o.name ORDER BY highest_risk_score DESC NULLS
LAST;
```

[TABLE: DOCUMENT] [SECTION: rag_retrieval_hints_query_rules_and_glossary] [DOC: compliance_platform_schema_v3]

17. RAG Retrieval Hints, Query Rules & Glossary

Structured metadata for the retrieval and query generation components of the RAG pipeline.

17.1 Natural Language to Table Mapping

Natural Language Keywords / Phrases	Primary Table	Common Join Tables
organization, company, client, tenant, org	organizations	—
user, person, employee, owner, who, auditor, analyst	users	organizations
framework, ISO 27001, SOC 2, HIPAA, standard, norm	frameworks	—
control, requirement, clause, domain, safeguard	controls	frameworks
policy, procedure, document, written policy, review date	policies	controls, users, organizations
audit, engagement, assessment, review, certification	audits	organizations, frameworks, users
finding, gap, deficiency, non-conformity, breach	audit_findings	audits, controls
risk, threat, exposure, likelihood, impact, risk score	risks	organizations, controls, users
task, action item, remediation, ticket, overdue, SLA	tasks	users, audit_findings, risks
evidence, proof, file, upload, attachment, document	evidence	tasks, audits, users

17.2 Critical Query Generation Rules

TENANT ISOLATION ALWAYS scope: WHERE org_id = (SELECT org_id FROM organizations WHERE name = 'X'). Never return cross-tenant data.

GENERATED COLUMN RULE risks.risk_score CANNOT be set via INSERT/UPDATE. Only use in SELECT / WHERE / ORDER BY. Attempting to write it raises a PostgreSQL error.

OPEN ITEM FILTERS findings: status IN ('open','in_remediation') | risks: status = 'open' | tasks: status IN ('todo','in_progress')

- Overdue tasks: WHERE due_date < CURRENT_DATE AND status != 'done'
- Policy review overdue: WHERE review_date < CURRENT_DATE AND status = 'active'
- evidence holds METADATA only — no file content is stored in the DB.
- A finding being 'closed' does NOT mean its parent audit is complete — check audits.status separately.
- Use LEFT JOIN when aggregations should include organizations with zero findings/risks/tasks.
- Score bands for risks: Critical >= 20 | High 15-19 | Medium 10-14 | Low 5-9 | Very Low < 5.

17.3 Glossary

Term	Definition
org_id	Tenant identifier. Present in most tables to enforce per-organization data isolation.

Term	Definition
framework_id	References a compliance standard (ISO 27001, SOC 2, HIPAA, etc.).
control_id	Atomic compliance requirement belonging to one framework.
risk_score	Auto-calculated as likelihood * impact. Range 1-25. GENERATED column — read-only.
finding	A documented deficiency raised during an audit against a specific control.
evidence	Metadata record for an uploaded file proving compliance or task completion. No file content stored.
owner_id	User accountable for a policy or risk. Not necessarily the creator.
auditor_id	User (auditor role) who conducts an audit engagement.
assigned_to	User assigned to complete a remediation task.
uploaded_by	User who uploaded an evidence file — accountability trail.
review_date	Date by which a policy must be re-reviewed and re-approved.
SERIAL	PostgreSQL auto-incrementing integer PK type. Never null, never manually set.
GENERATED ALWAYS AS STORED	PostgreSQL computed column. Derived from other columns, physically stored. Cannot be set via INSERT/UPDATE.
CHECK constraint	DB-level validation restricting a column to a specific set of allowed values.
DEFAULT NOW()	Automatically sets a TIMESTAMP to the current DB time at INSERT.
NULLABLE FK	A foreign key column that may be NULL — the relationship is optional.